Solutions: Day 34 Practice Problems

Complete, production-tested solutions and architectural walkthroughs for Day 34 terminal challenges.

Solution 1: Production Server Triage & Path Resolution

1. Print Current Location

```
pwd
```

- **Explanation**: `pwd` stands for **Print Working Directory**. It queries the OS process table and returns the full absolute path of your current shell process (e.g., `/home/engineer`).

2. Detailed Directory Listing (Including Hidden Dotfiles)

```
ls -la
# Or equivalently:
ls -l -a
```

- **Explanation**:
  - `-l`: Enables the long listing format displaying permissions (`drwxr-xr-x`), owner, group, file size in bytes, and last modification timestamp.
  - `-a`: Instructs `ls` to display all files including dotfiles (`.`, `..`, `.bashrc`, `.env`). In Unix/Linux, any filename prefixed with a period (`.`) is considered hidden.

3. Absolute vs Relative Navigation

- **Path Classification**:
  - `/var/log/cv_pipeline/crash_dump.log` is an **Absolute Path** because it begins with the root directory slash (`/`). It unambiguously points to the exact same file regardless of what your current working directory is.
- **Navigate to Logs using Absolute Path**:

  ```
  cd /var/log/cv_pipeline
  ```

- **Navigate Up Two Levels to `/var/` using Relative Path**:

```
cd ../..
```

- **Explanation**: `.` represents current directory; `..` represents the parent directory. Moving from `/var/log/cv_pipeline` up one level (`..`) reaches `/var/log`. Chaining a second `..` (`../..`) reaches `/var`. Because this command does not begin with `/`, it is resolved relative to the current working directory.

4. Return to Home Directory

```
cd ~
# Or simply:
cd
```

- **Explanation**: The tilde symbol `~` is automatically expanded by Bash to the environment variable `$HOME` (e.g., `/home/engineer`). Invoking `cd` with zero arguments defaults to navigating directly to `$HOME`.

Solution 2: MLOps Project Workspace Scaffolding

1. Directory Tree Creation

```
# Step 1: Create project root and enter it
mkdir sentiment_llm
cd sentiment_llm

# Step 2: Create subdirectories
mkdir data src checkpoints

# Step 3: Create nested subdirectories in data
mkdir data/raw data/processed
```

Alternative one-liner utilizing the parent flag `-p`:

```
mkdir -p sentiment_llm/{src,checkpoints,data/{raw,processed}}
cd sentiment_llm
```

2. File Initialization via `touch`

```
# Create source files inside src
touch src/dataset.py src/model.py src/train.py
```

```
# Create manifest and hidden config file in project root
touch requirements.txt .env
```

- **Explanation**: `touch` creates 0-byte empty files instantly if they do not exist. Passing multiple filename arguments instructs the shell to initialize all listed files in a single system call.

3. Timestamp Refresh

```
touch requirements.txt
```

- **Explanation**: When `touch` is run on a file that already exists, it leaves the contents and size completely intact, but updates the file's Access Time (`atime`) and Modification Time (`mtime`) to the current system clock. Build systems like `make` or caching daemons rely on `mtime` to detect pipeline updates.

4. Verification Command

```
ls -la
ls -la src/
ls -la data/
```

- **Output Verification**:

```
drwxr-xr-x 2 engineer engineer 4096 Oct 28 00:20 .
drwxr-xr-x 4 engineer engineer 4096 Oct 28 00:19 ..
-rw-r--r-- 1 engineer engineer    0 Oct 28 00:20 .env
drwxr-xr-x 2 engineer engineer 4096 Oct 28 00:20 checkpoints
drwxr-xr-x 4 engineer engineer 4096 Oct 28 00:20 data
-rw-r--r-- 1 engineer engineer    0 Oct 28 00:20 requirements.txt
drwxr-xr-x 2 engineer engineer 4096 Oct 28 00:20 src
```

Solution 3: Defensive Disk Cleanup & Safe Deletion Governance

1. Empty vs Non-Empty Directory Deletion

- **Safely Deleting Empty Directory**:

```
rmdir stale_cache
```

    - **Behavior**: If `stale_cache` is empty, it is deleted cleanly. If it contains even one hidden file or subdirectory, `rmdir` aborts immediately with: `rmdir: failed to remove 'stale_cache': Directory not empty` This makes `rmdir` the safest tool for automated garbage collection scripts.
- **Deleting Non-Empty Directory Tree**:

```
rm -rf run_epoch_01
```

    - **Behavior**: `-r` (recursive) descends through all child directories and files. `-f` (force) suppresses confirmation prompts and ignores non-existent files.

2. Risk Analysis of Dangerous Command Patterns

- **The Accidental Space Catastrophe**:

```
rm -rf / tmp/old_checkpoints
```

    - **What Happened**: Because of the stray space between `/` and `tmp`, the shell parses this as two separate target arguments:
        1. Target 1: `/` (The Root Directory of the operating system!)
        2. Target 2: `tmp/old_checkpoints`
    - **Consequence**: The shell attempts to recursively delete every single file, kernel module, configuration file, binary (`/bin`, `/etc`, `/usr`, `/lib`), and connected volume on the server. The entire server crashes and cannot be rebooted. Modern Linux versions include `--no-preserve-root` protection, but older systems or subpath typos (e.g. `rm -rf /opt /app`) cause catastrophic data loss.
- **Why Terminal Deletion Bypasses the Recycle Bin**:
    - The Desktop Recycle Bin / Trash is not an OS file deletion; it is a user-space utility that simply moves files to a hidden folder (e.g., `C:\$Recycle.Bin` or `~/.local/share/Trash`).
    - Terminal commands `rm` invoke the POSIX `unlink()` system call directly. The inode link count is decremented to 0, the disk allocation blocks are marked as free in the filesystem superblock, and metadata pointers are purged. No GUI restoration is possible.

3. Production Defensive Workflow

Before running any destructive `rm` command on servers:

1. **Verify Location**: Run `pwd` to ensure you are in the exact intended directory.
2. **Dry-Run / List First**: Replace `rm` with `ls` to visually inspect what matches:

```
ls run_epoch_*
```

3. **Target Specificity & Prompting**: Avoid wildcards on root paths, and test with interactive prompt `-i` if uncertainty exists:

```
rm -ri run_epoch_01/
```\n